Disciplina: Construção de Páginas Web II
Professora: Milena Alegre

LISTA 01

Objetivo
Utilizando como base o Card do Herói desenvolvido na atividade anterior, implemente as funcionalidades solicitadas abaixo. A dupla deverá evoluir o mesmo projeto ao longo das etapas.

Parte 1 – Expandindo o formulário

1. Nome do Herói

Utilize <input type="text">; campo obrigatório; mínimo de 3 e máximo de 20 caracteres.

2. Classe

Utilize <select> com pelo menos 4 opções: Guerreiro, Mago, Arqueiro e Curandeiro. A seleção é obrigatória.

3. Nível

Utilize <input type="number"> com valor mínimo 1 e máximo 100.

4. E-mail do jogador

Utilize o tipo correto de input e torne o campo obrigatório.

5. Código do Herói

Utilize pattern. Formato obrigatório: ABC-123 (3 letras maiúsculas + hífen + 3 números).

6. Senha

Mínimo de 8 caracteres, contendo pelo menos uma letra maiúscula, uma minúscula e um número.

Parte 2 – Feedback visual

7. Utilize CSS para diferenciar visualmente campos válidos e inválidos.

Podem ser utilizadas :valid/:invalid ou :user-valid/:user-invalid.

8. Crie pelo menos uma mensagem de erro personalizada usando .

A mensagem deverá aparecer quando algum campo não estiver de acordo com a regra definida.

Parte 3 – Validação com JavaScript

9. O formulário deve ser acessado via document.getElementById().

10. Utilize .value para acessar os valores digitados ou selecionados.

11. Antes de criar o herói, utilize checkValidity().

Se o formulário estiver válido, permita a criação. Caso contrário, não permita.

12. Quando o formulário estiver inválido, utilize reportValidity() para indicar ao usuário o campo que precisa ser corrigido.

Parte 4 – Eventos

13. Nome em tempo real

Ao digitar o nome, atualize automaticamente o nome apresentado no Card. Utilize o evento input.

14. Classe em tempo real

Ao alterar a classe no <select>, atualize automaticamente o Card. Utilize o evento change.

15. Validação ao sair do campo

Escolha um campo e faça uma validação quando ele perder o foco. Utilize o evento blur.

Parte 5 – Card do Herói

16. Crie a função atualizarDados().

Ela deverá atualizar no Card: nome, classe, nível e cor escolhida, utilizando manipulação da DOM.

Parte 6 – Objeto JavaScript

17. Crie a função capturarDados().

A função deverá capturar os dados do formulário e organizá-los em um objeto JavaScript.

Parte 7 – Segunda página do personagem

18. Crie uma nova página chamada detalhes.html.

Na primeira página (index.html), crie o botão PRÓXIMA ETAPA.

Antes de avançar, verifique se os dados obrigatórios estão válidos.

Se estiverem corretos, direcione para detalhes.html usando window.location.href. Se houver erro, permaneça na página atual.

19. Na página detalhes.html, adicione novos dados para complementar o personagem.

Arma principal – <select>.

Quantidade de vida – <input type="number">, mínimo 0 e máximo 100.

Quantidade de mana – <input type="number">, mínimo 0 e máximo 100.

Data de criação do personagem – <input type="date">.

Descrição / História – <textarea>, com no mínimo 20 caracteres.

Parte 8 – Formulário em etapas e navegação

20. Na página detalhes.html, crie um botão VOLTAR.

Ao clicar, retorne para index.html utilizando window.location.href.

21. Crie também um botão FINALIZAR PERSONAGEM.

Antes de finalizar, valide os campos da segunda página utilizando recursos como required, min, max, minlength e checkValidity().

22. Ao finalizar, mostre na própria página um resumo dos dados preenchidos na segunda etapa.

Exiba arma, vida, mana, data de criação e história utilizando manipulação da DOM.

23. Desafio: faça com que pelo menos um dado da primeira página apareça na segunda página.

Exemplo: nome e/ou classe do herói.

A passagem da primeira para a segunda etapa deverá ocorrer somente após a validação dos campos obrigatórios.

A dupla poderá utilizar uma forma de transporte de dados entre páginas orientada pela professora.

Parte 9 – Organização do código

24. Organize o código em funções com responsabilidades claras.

Mantenha capturarDados(), atualizarDados() e outras funções necessárias.

25. Utilize addEventListener() em pelo menos 3 situações diferentes no projeto.

26. Não coloque todo o JavaScript diretamente no HTML. Mantenha o código organizado em script.js.

Parte 10 – Questões Teorica

No arquivo README.md, respondam:

27. Qual a diferença entre required e pattern?

28. Qual a diferença entre checkValidity() e reportValidity()?

29. Para que serve .value?

30. Qual a diferença entre os eventos input, blur e change?

31. Explique o que acontece em document.getElementById("nome").value.

32. Qual a função do addEventListener()?

33. Por que a validação deve acontecer antes de avançar para a próxima página?

Entrega

A dupla deverá enviar os arquivos compactados contendo:

index.html

detalhes.html

style.css

script.js

README.md

O projeto deverá estar organizado e funcional.

Critérios de avaliação

Critério	Valor
Estrutura HTML e formulário	1,0
Validações HTML (required, pattern, min, max etc.)	1,5
Validação e eventos com JavaScript	2,0
Manipulação do Card / DOM	1,5
Navegação entre páginas e formulário em etapas	1,5
Organização das funções e do código	1,0
CSS e feedback visual	0,5
README e explicação das questões	1,0
TOTAL	10,0

Bom Trabalho! 🙂